DACC | Departamento Acadêmico de Ciência da Computação

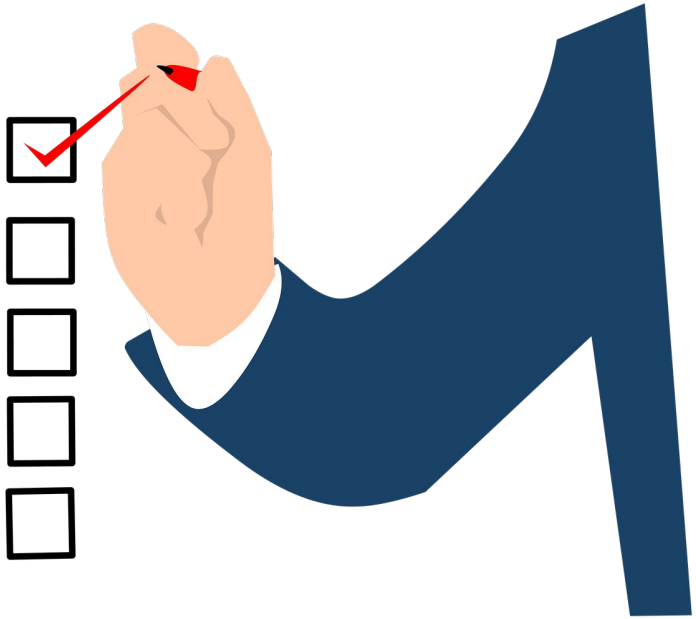
FUNDAÇÃO UNIVERSIDADE FEDERAL DE RONDÔNIA

Programação Orientada a Objetos

Professor:
Dr. Lucas Marques da Cunha
lucas.marques@unir.br

Roteiro

1. Apresentação
2. Horário
3. Objetivos e Conteúdos
4. Metodologia de Avaliação
   a. Lista de Exercícios
5. Cronograma de Aulas
6. Dúvidas e/ou sugestões
7. POO: Visão Geral



Apresentação

- Formação
  - Bacharel em Sistemas de Informação - UNIFIP (2012)
  - Especialista em Desenvolvimento Web e Mobile - IFSUDESTMG (2026)
  - Mestre em Informática - UFPB (2016)
  - Doutor em Bioinformática - UFRN (2022)
- Linhas de Pesquisa
  - Processamento de Imagens Digitais: Análise Forense
  - Inteligência Artificial e Aprendizagem de Máquina
  - Softwares Educacionais/Desenvolvimento para dispositivos móveis
  - Bioinformática: proteômica, genômica e banco de dados biológicos
  - Linguagens de programação
    - Perl, Java, C, Octave/Matlab e R, Android Embarcado.
- Atuação
  - Unidade de Educação a Distância - UFPB (2014 - 2021)
  - Instituto de Educação Profissional - INEP (2014 - 2016)
  - UNINASSAU (2019 - 2021)

Horário

COMPONENTE CURRICULAR: Programação Orientada a Objetos

PERÍODO: 3º

CARGA HORÁRIA: 100H

HORÁRIO:

Presencial:
Quinta-feira (13h50min - 18h10min) - Tarde
Assíncrona:
Sábado (13h50min) - Tarde

Objetivos e Conteúdos

Introduzir os conceitos de programação orientada a objetos e metodologia de desenvolvimento de software segundo esse paradigma.

Programação orientada a objetos: objetos, classes, herança, polimorfismo e interfaces. Tratamento de exceções. Abstrações, generalizações, super e sub-classes e instâncias. Ocultamento. Agregações como listas, conjuntos e arranjos. Interface gráfica com o usuário e tratamento de eventos.

Metodologia de avaliação

1. Durante o período o letivo serão realizadas 02 (duas) avaliações.
2. **A primeira a avaliação será composta de:**
   a. Prova Teórica (peso 4,0)
   b. Prova Prática (peso 4,0)
   c. Lista de Exercícios (peso 2,0)
3. **A segunda avaliação será composta de:**
   a. Prova Teórico-Prática (Peso 4,0)
   b. Projeto Final (Peso 4,0)
   c. Lista de Exercícios (Peso 2,0)

4. Se **média final < 60** o aluno fará avaliação repositiva nos termos regimentais da UNIR.
5. A **avaliação repositiva** irá substituir o menor das notas (N1 ou N2). Então se calculará novamente a **média final**.

Cronograma de aulas

UNIR

DACC Departamento Acadêmico de Ciência da Computação
FUNDAÇÃO UNIVERSIDADE FEDERAL DE RONDÔNIA

Programação Orientada a Objetos

Aula 01: Uma Nova Forma de Pensar Software

Departamento Acadêmico de Ciência da Computação - UNIR
Professor: Dr. Lucas Marques da Cunha
Email: lucas.marques@unir.br

NotebookLM

Fundamentos do Paradigma OO

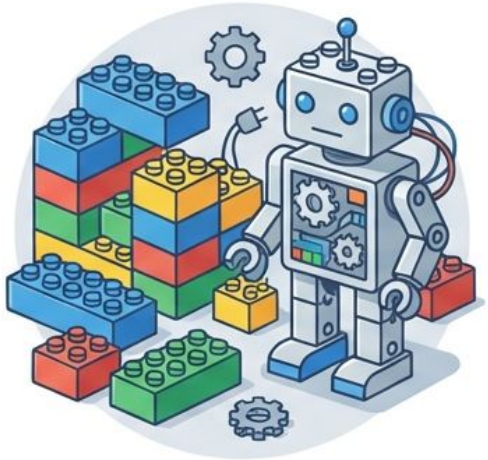
O que é um Paradigma de Programação?

É um estilo que **guia** a forma como os programas são escritos e estruturados. Não é apenas código; é uma mentalidade.

Você já pensou em como os objetos do mundo real podem ser representados em um computador?

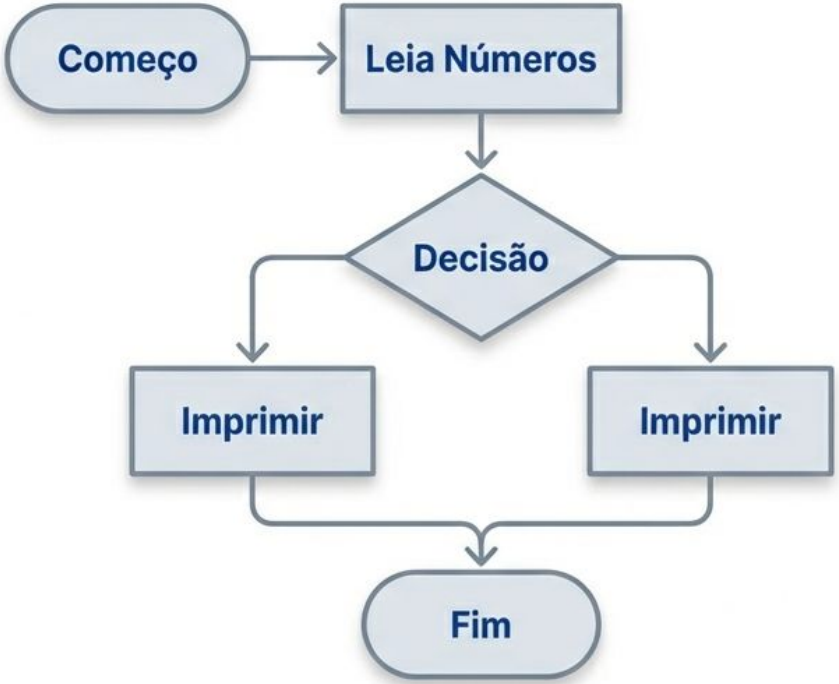
Você percebe a semelhança entre montar LEGO e criar software?

Você consegue identificar conceitos de POO nos aplicativos que usa hoje?



NotebookLM

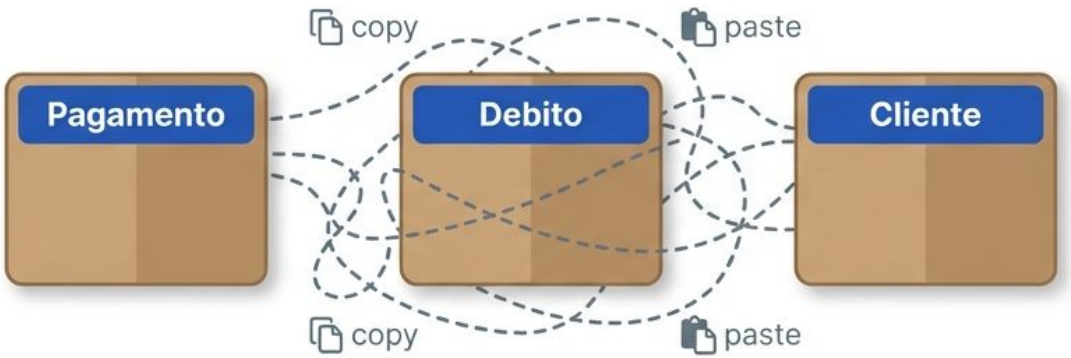
O Paradigma Estruturado



- **Ênfase em Procedimentos:** Sequências de instruções para realizar tarefas.
- **Decomposição:** Quebra o problema em tarefas menores (funções).
- **Estruturas Básicas:** Sequência, Decisão (if/else), Iteração (loops).
- **Separação:** Dados e Funções vivem em lugares separados.

NotebookLM

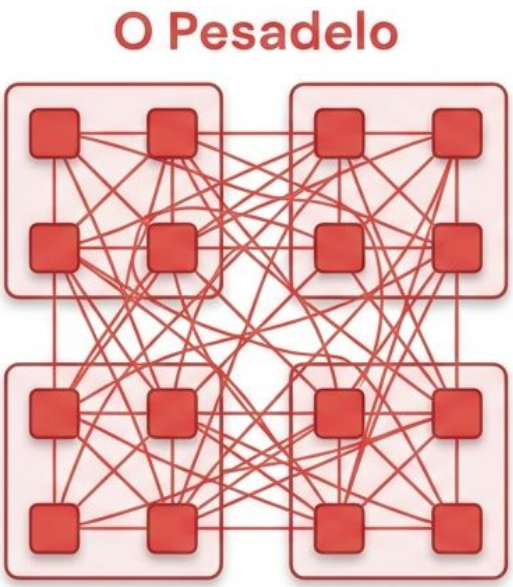
O Problema da Compnelexidade



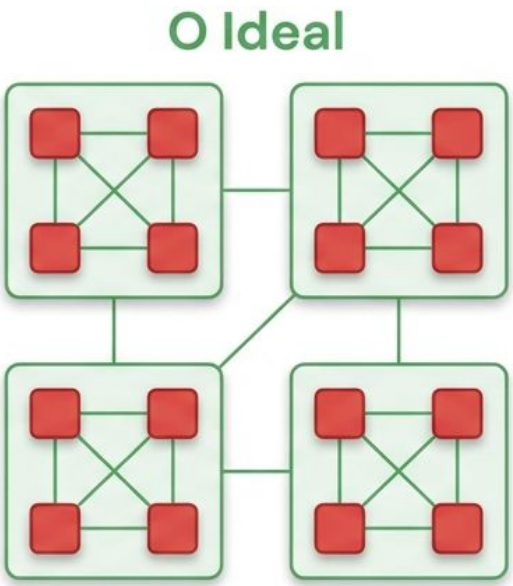
- **O Limite:** Funciona bem para uma calculadora básica, mas e para um Sistema de Gestão Escolar complexo?
- **A Dificuldade:** Em linguagens como C, reutilizar dados e comportamentos exige 'copiar e colar' estruturas ou criar conexões complexas e frágeis.

NotebookLM

Os Inimigos de um Bom Software



Baixa Coesão / Alto Acoplamento



Alta Coesão / Baixo Acoplamento

Acoplamento: O nível de interdependência entre os códigos. (Queremos pouco!)

Coesão: Uma unidade deve ser responsável apenas por tarefas do seu próprio conceito. (Queremos muito!)

NotebookLM

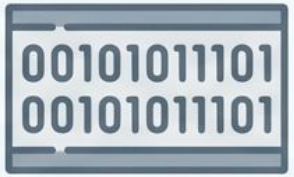
Fundamentos do Paradigma OO

A Solução: Orientação a Objetos

Objeto (Classe)

Dados:
Nome, CPF

Comportamento:
Pagar, Andar



C-struct
Código Binário

O intuito da POO é aproximar o manuseio do software ao manuseio das coisas do mundo real. É um paradigma baseado na **composição e interação** entre unidades chamadas **Objetos**.

NotebookLM

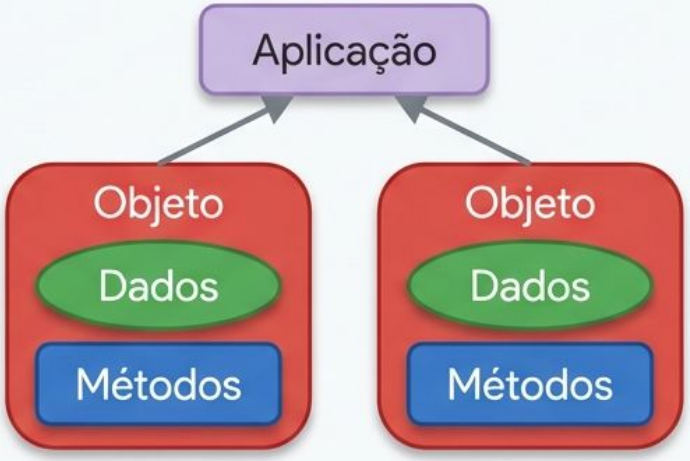
Fundamentos do Paradigma OO

Comparando as Estruturas

Estruturada

Dados Globais + Procedimentos

Orientada a Objetos



Encapsulamento e Interação

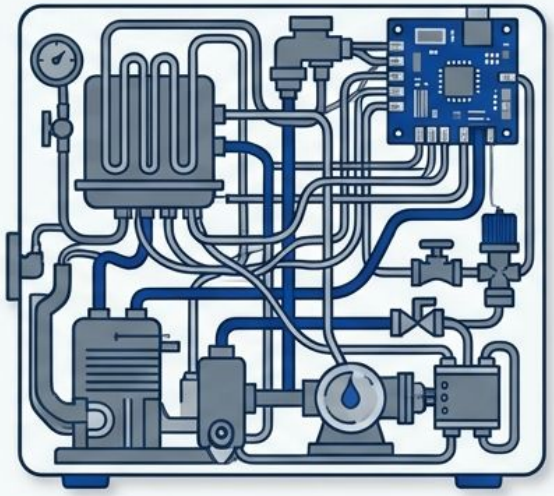
Na POO, dados e comportamentos estão **encapsulados** juntos.

NotebookLM

Os 4 Pilares da POO

NotebookLM

1. Abstração



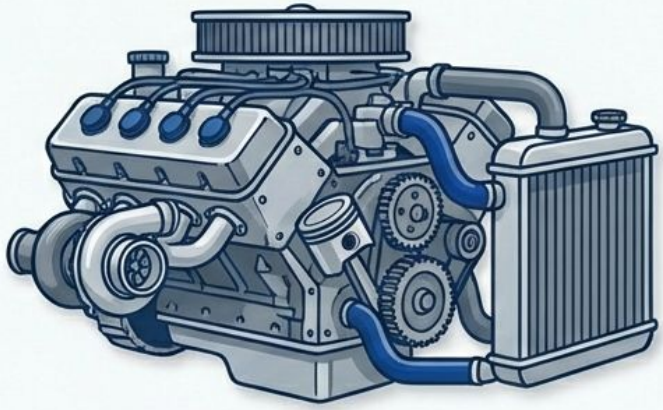
O Conceito: Isolar as características essenciais de um objeto, ignorando detalhes complexos desnecessários para o usuário.

Exemplo Prático: Para fazer café, você quer apenas apertar um botão (**Interface**). Sem abstração, você teria que controlar manualmente a temperatura da água, a pressão e o fluxo (**Implementação**).

NotebookLM

2. Encapsulamento

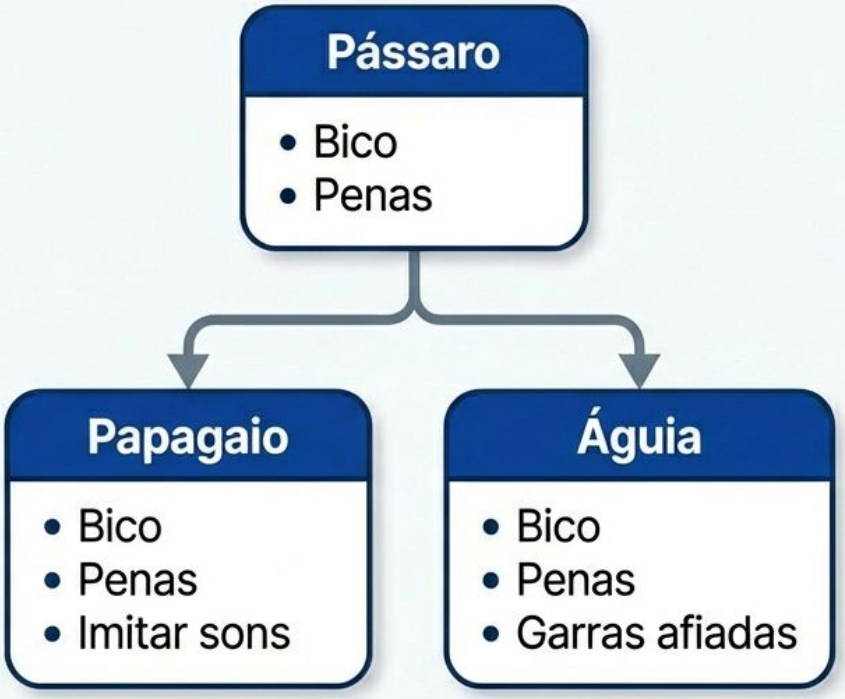
Public Interface (Dashboard) → **Private Implementation** (Engine)



- **O Conceito:** Ocultar detalhes internos de implementação e proteger os dados.
- **A Regra:** O que é interno é **Privado**; o que é acessível é **Público**.
- **Benefício:** Segurança e facilidade de manutenção. Se o motor for trocado, o motorista não precisa aprender a dirigir novamente.

NotebookLM

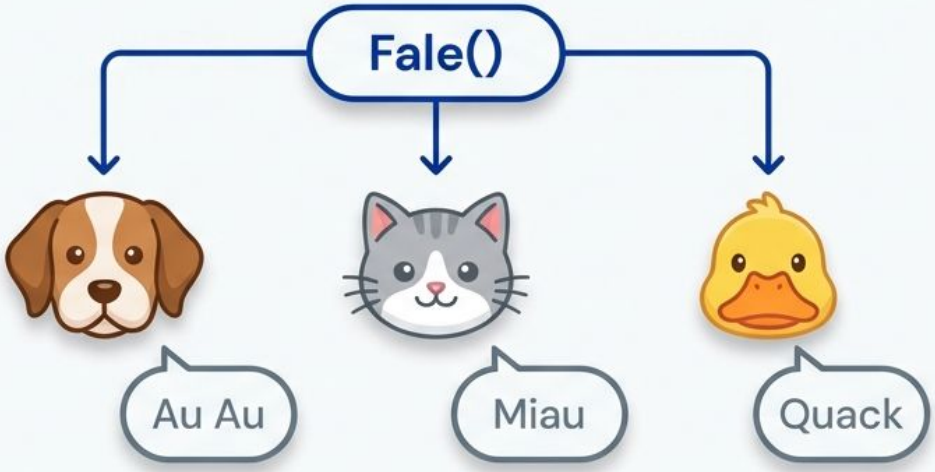
3. Herança



- **O Conceito:** Permite que um objeto adquira propriedades e métodos de outro.
- **O Objetivo: Reúso.** Evita a repetição de código.
- **Lógica:** O 'Filho' estende o 'Pai', mantendo alta coesão.

NotebookLM

4. Polimorfismo



- **O Conceito:** A capacidade de objetos de diferentes classes responderem ao mesmo método de formas diferentes.
- **Resultado:** O mesmo comando gera comportamentos específicos para cada tipo de objeto.

NotebookLM

A Ferramenta: Java

Tech Profile: Java

Origem: 1991, Sun Microsystems (Green Project).

Criadores: James Gosling, Patrick Naughton e Mike Sheridan.

Características Chave:

- **Portável:** "Write Once, Run Anywhere" (Independente de plataforma).
- **Tipada:** Segurança na definição de variáveis.
- **Orientada a Objetos:** Projetada desde o início para este paradigma.

NotebookLM

Fundamentos do Paradigma OO

Por Que Adotar POO?

Simplicidade:
Modela o mundo real (Fecha o Gap Semântico).

Manutenção:
Alta Coesão e Baixo Acoplamento facilitam ajustes.

Reúso:
Herança e Encapsulamento economizam tempo.

Segurança:
Protege dados sensíveis contra acesso indevido.

POO não é apenas sobre escrever código; é sobre projetar soluções sustentáveis.

NotebookLM

DACC Departamento Acadêmico de Ciência da Computação
FUNDAÇÃO UNIVERSIDADE FEDERAL DE RONDÔNIA

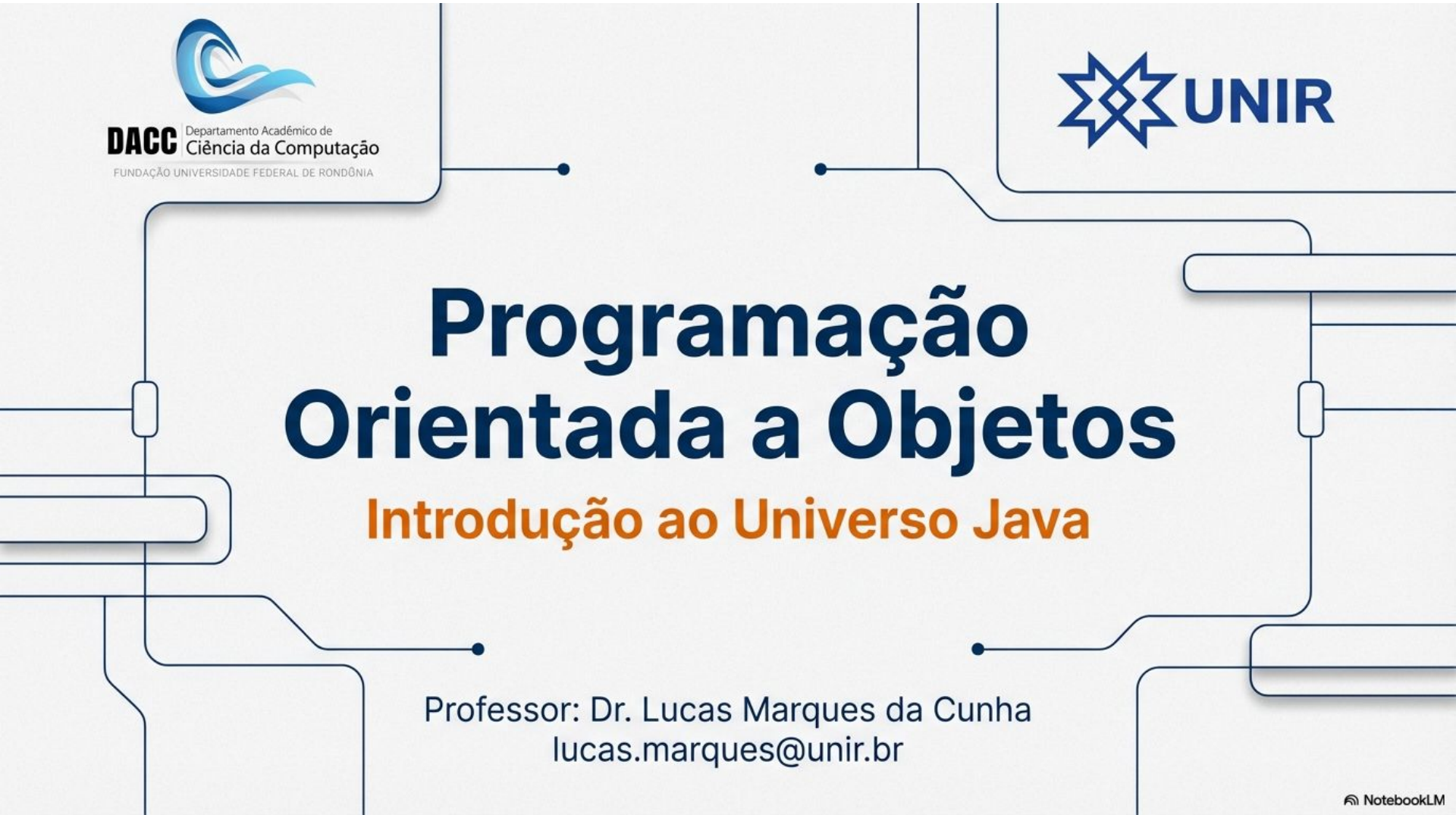
Obrigado!

Dúvidas? Entre em contato.

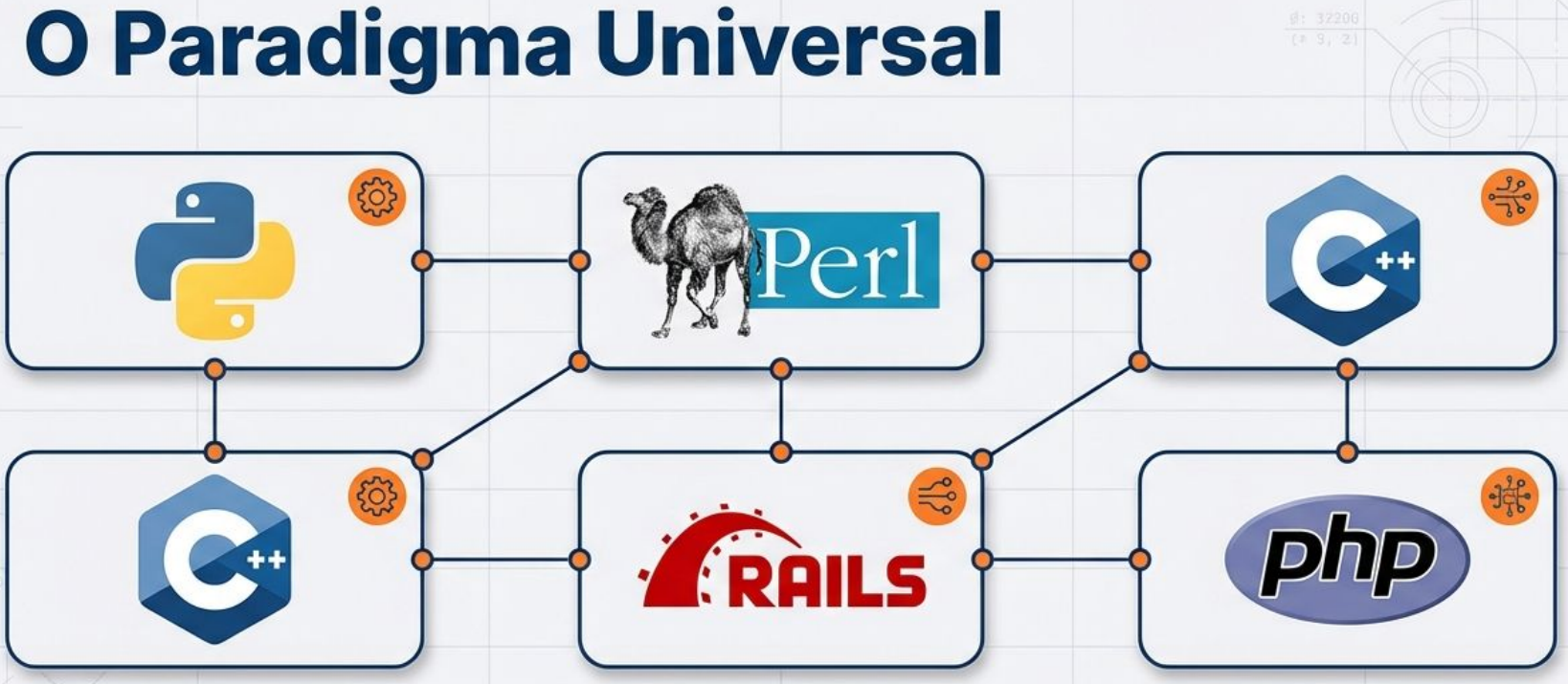
Professor: Dr. Lucas Marques da Cunha
Email: lucas.marques@unir.br
Departamento Acadêmico de Ciência da Computação

Fonte: materialpublic.imd.ufrn.br

NotebookLM


DACC Departamento Acadêmico de Ciência da Computação
FUNDAÇÃO UNIVERSIDADE FEDERAL DE RONDÔNIA
UNIR
Programação Orientada a Objetos
Introdução ao Universo Java
Professor: Dr. Lucas Marques da Cunha
lucas.marques@unir.br
NotebookLM

Fundamentos do Paradigma OO



A Orientação a Objetos não é exclusividade de uma linguagem. É um conceito fundamental que estrutura a tecnologia moderna, do web development a sistemas corporativos.

NotebookLM

Fundamentos do Paradigma OO

UNIR

Mesmo Conceito, Diferentes Sotaques

Perl

```perl
sub new {
    my ($class) = shift;
    my $self = {
        _nome => shift,
        _sobrenome => shift
    };
    bless $self, $class;
    return $self;
}
```

Python

```python
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```

C#

```csharp
public class Taxi
{
    public bool IsInitialized;
    public Taxi()
    {
        IsInitialized = true;
    }
}
```

Todas estas linguagens definem **Construtores** para iniciar objetos.
A lógica é a mesma; apenas a sintaxe muda.

NotebookLM

UNIR

Java: A Origem

1991
Início na Sun Microsystems.

Green Project
Focado originalmente em dispositivos móveis e eletrônicos de consumo.

Criadores
James Gosling, Patrick Naughton e Mike Sheridan.

Nascida como uma linguagem para eletrônicos inteligentes, Java evoluiu para se tornar a espinha dorsal da internet corporativa.

NotebookLM

Fundamentos do Paradigma OO

UNIR

As Superpotências da Linguagem

Orientada a Objetos

Modelagem próxima do mundo real.

Portabilidade

Inter Regular, Escreva uma vez, execute em qualquer lugar.

Segurança

Ambiente Ambiente controlado de execução.

Simples & Tipada

Detecta erros na declaração, facilitando a manutenção.

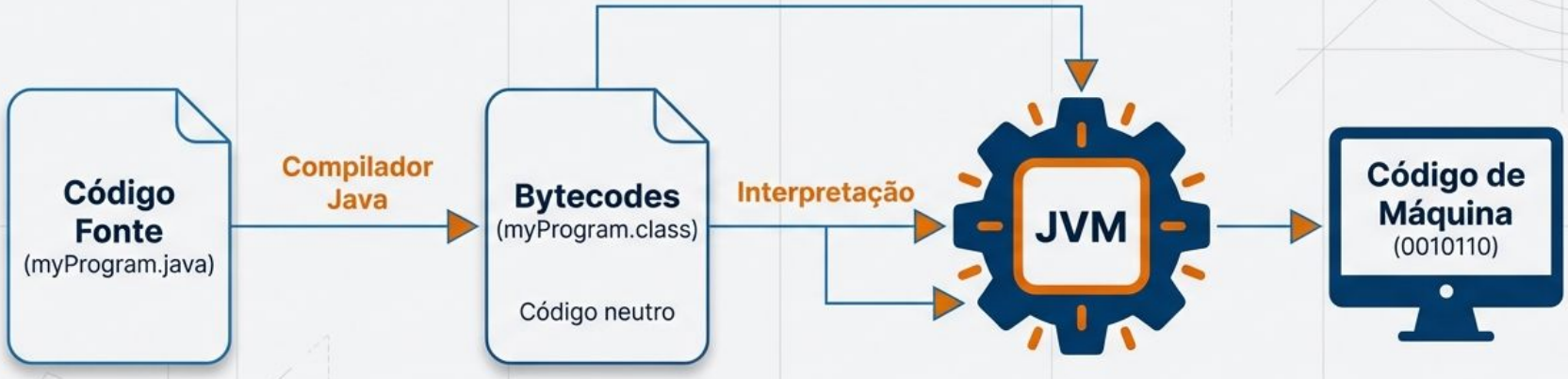
Distribuída

Pronta para Pronta para redes e internet.

NotebookLM

UNIR

Sob o Capô: A Máquina Virtual Java (JVM)



O segredo da portabilidade: O compilador gera **Bytecodes** universais. A **JVM** traduz esses códigos para a linguagem específica do seu computador, seja ele Windows, Mac ou Linux.

NotebookLM

Relevância de Mercado em 2024

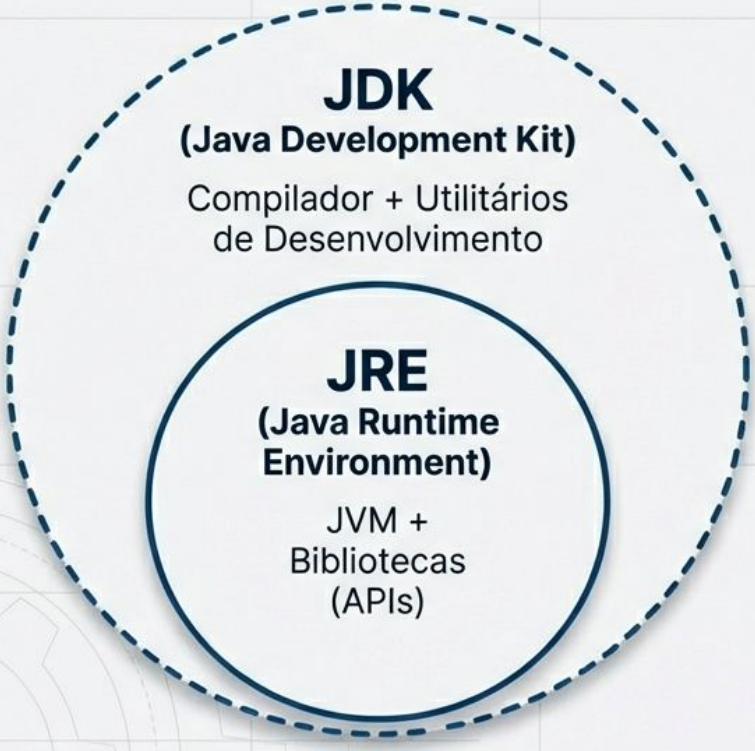
"Java permanece consistentemente no topo dos rankings globais de popularidade e demanda de emprego."

Dados baseados em LinkedIn Pulse e Techlise analysis.

NotebookLM

UNIR

Sua Caixa de Ferramentas



JRE: Para quem quer **EXECUTAR**. Contém apenas o necessário para rodar programas.

JDK: Para quem vai **CRIAR**. Contém o JRE mais as ferramentas para escrever código.

Como desenvolvedor, você precisará baixar e instalar o **JDK**.

NotebookLM

O Ambiente de Desenvolvimento: Eclipse

eclipse

IDE Robusta

Ambiente de Desenvolvimento Integrado feito em Java.

Open Source

Código aberto e gratuito.

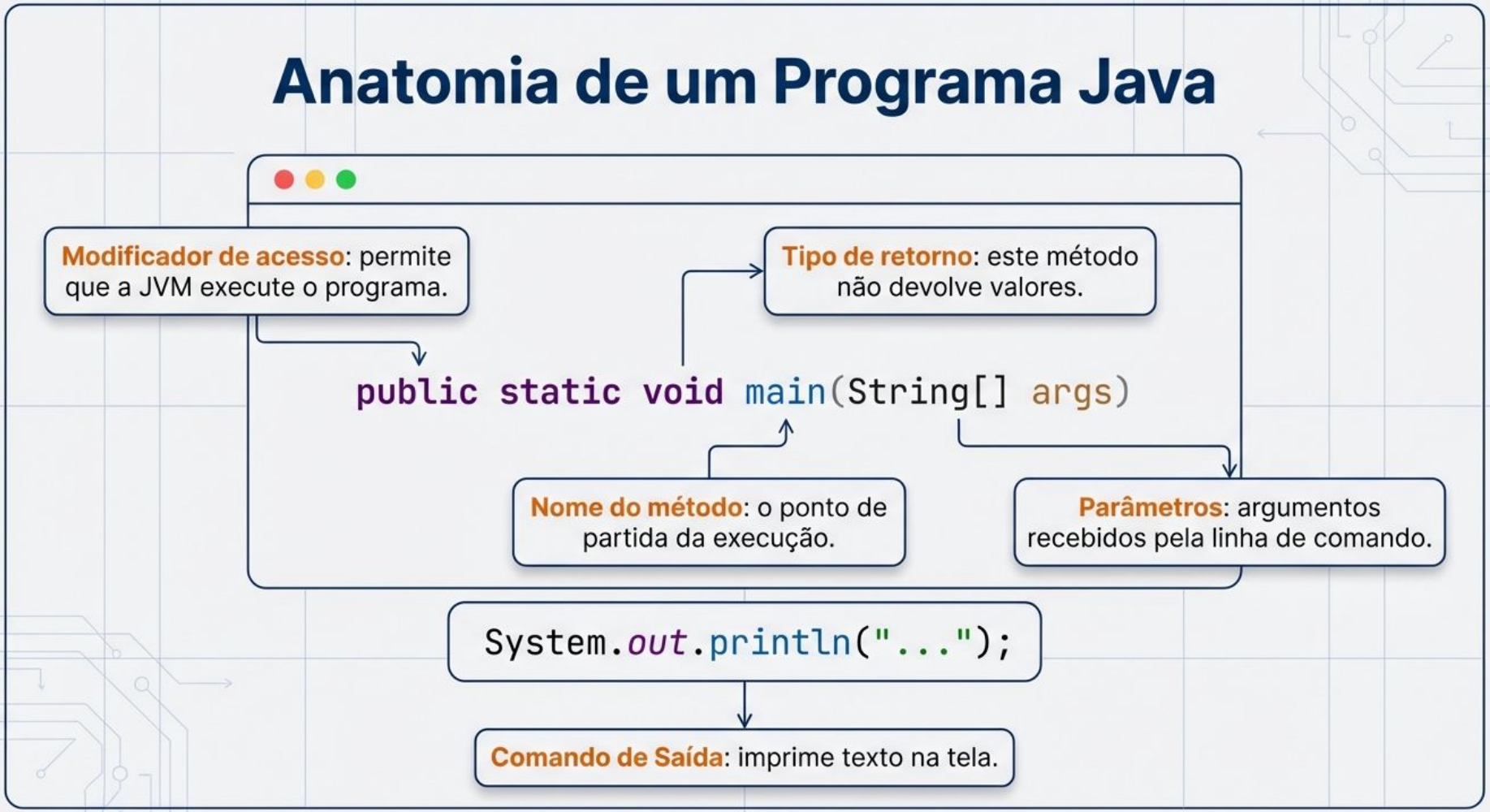
Versatilidade

Suporta C/C++, PHP, Python e Android via plugins.

Disponível em **eclipse.org**

NotebookLM

Fundamentos do Paradigma OO



NotebookLM

UNIR

Seu Primeiro Programa

```
public class HelloUniverse {
   public static void main(String[] args) {
      System.out.println("Hello Universe!");
   }
}
```

O método **main** é o ponto de partida. Sem ele, a aplicação Java não começa a rodar.

```
> Hello Universe!
```

NotebookLM

UNIR

Interatividade: Entrada de Dados

C: `scanf()`
Pascal: `readln()`

→ **Scanner**

```
import java.util.Scanner;

Scanner input = new Scanner(System.in);
```

Importação do pacote utilitário

Criação do objeto de leitura conectado ao teclado

A classe **Scanner** é a responsável por ler o que o usuário digita no console.

NotebookLM

Lógica em Ação: Somando Números

Import: Traz a ferramenta Scanner.

Input: `nextInt()` captura números inteiros.

Process: Operador `+` realiza a soma.

Output: `printf` formata o resultado.

```java
import java.util.Scanner;

public class Addition {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        int number1;
        int number2;
        int sum;

        System.out.print("Enter first integer: ");
        number1 = input.nextInt(); // Lê primeiro número

        System.out.print("Enter second integer: ");
        number2 = input.nextInt(); // Lê segundo número

        sum = number1 + number2; // Soma

        System.out.printf("Sum is %d%n", sum); // Exibe
    }
}
```

NotebookLM

UNIR

Resumo da Jornada

- **Contexto OO**: Java compartilha conceitos fundamentais com C++, C# e Python.
- **A JVM**: A máquina virtual que garante a independência de plataforma.
- **Ferramentas**: JDK para desenvolver, Eclipse como ambiente de trabalho.
- **Sintaxe Básica**: Estrutura de classes, método `main` e uso de `System.out`.
- **Input**: Uso da classe `Scanner` para interagir com o usuário.

NotebookLM

Dúvidas ou Sugestões?

lucas.marques@unir.br

NotebookLM